Determinación del umbral de percolación de una mezcla de material conductor

José Antonio Quinto Ciocca
Universidad Nacional Mayor de San Marcos, Lima Perú
joseantonio.quinto@unmsm.edu.pe

10 de julio de 2024

Resumen

En la actualidad los cementos inteligentes y en particular el cemento conductor está tomando relevancia gracias a sus amplios usos. Para fabricar un cemento conductor se mezcla el cemento con materiales conductores como el carbón. El umbral de percolación es la concentración mínima de material conductor en la mezcla de manera que este se vuelva conductor. El comportamiento hidrofóbico del carbón forma redes conductoras dentro del material, la mezcla será conductora si existe un camino que lleve de un lado a otro de la matriz, para simular este proceso consideramos que las partículas conductoras se mueven aleatoriamente y se agrupan formando clústeres que también se moverán aleatoriamente. Se realizó un código en Python para simular el proceso, posteriormente se buscó si existe un camino que vuelva al material conductor. Los resultados indicaron que con pocas iteraciones se forma una red conductora, y que el umbral de percolación tiene un comportamiento exponencial decreciente versus el tamaño de la matriz.

Palabras clave: Simulación, percolación, cemento conductor.

Determination of the percolation threshold of a mixture of conductive material

Abstract

Currently, smart cements and in particular conductive cement are gaining relevance thanks to their wide uses. To make conductive cement, cement is mixed with conductive materials such as coal. The percolation threshold is the minimum concentration of conductive material in the mixture so that it becomes conductive. The hydrophobic behavior of carbon forms conductive networks within the material, the mixture will be conductive if there is a path that leads from one side of the matrix to the other. To simulate this process we consider that the conductive particles move randomly and group together forming clusters that also They will move randomly. A code was made in Python to simulate the process, then it was searched to see if there is a path that returns to the conductive material. The results indicated that with few iterations a conductive network is formed, and that the percolation threshold has an exponentially decreasing behavior versus the size of the matrix.

Keywords: Simulation, percolation, conductive cement.

1. Introduction

En la actualidad se necesita que el cemento tenga otras propiedades para construir edificaciones con características que nos son de utilidad, de entre los cementos inteligente el cemento conductor ha

cobrado importancia debido a sus aplicaciones, estas van a desde mantener calientes habitaciones, hasta servir de detectores de incendios. [Savytskyi et al., 2024] Esto es posible ya que el cemento conductor posee una resistencia que lo calienta al pasar una corriente eléctrica, sirviendo no solo para mantener caliente un espacio, sino que también ha servido como una forma económica de mantener veredas y caminos libres de nieve en lugares fríos. Cuando el cemento se combina con el grafito, este tiene la propiedad de volverse más o menos conductor de acuerdo a la temperatura que lo rodea debido a la expansión interna de grafito que crea vías para los electrones. [Conductive, 2021] Se pueden monitorear estas variaciones para identificar incendios, evitando así las falsas alarmas de los detectores de humo. Para hacer el cemento conductor es necesario combinarlo con materiales conductores, de manera que esto no afecte de una manera importante a sus propiedades originales como su dureza, por ello es importante conocer el umbral de percolación, el cuál es la cantidad mínima con la cuál una mezcla de cemento-conductor se vuelve un material conductor [Obregón G. Jesús, 2019]. La percolación se alcanza cuando hay un contacto directo entre partículas dentro del cemento, es decir, es necesario que se forme una red conductora dentro del cemento. Los materiales que se usan ampliamente para fabricar el cemento conductor son el "coke", el grafito, el grafeno, las fibras de carbono y el negro de carbón. [Savytskyi et al., 2024]

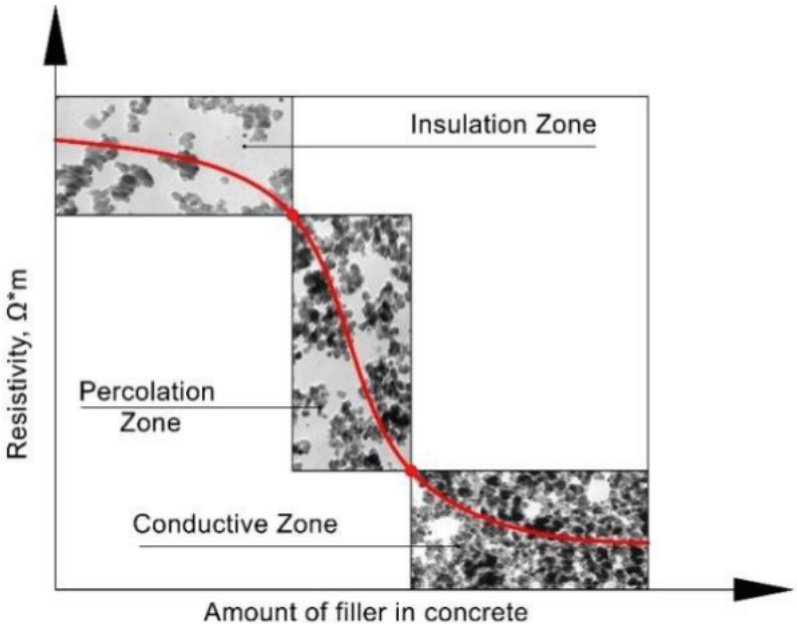


Figura 1: Umbral de percolación en el concreto

En pruebas experimentales se encontró que el umbral de percolación para el cemento era del 3 %-4 % [Chanut et al., 2023]. En la mezcla de cemento-negro de carbón, la interacción del cemento hidrofílico y el carbón hidrofóbico con el agua hace que se formen las redes conductoras dentro del material que vuelven conductor al cemento. Se intentará realizar una simulación simplificada del comportamiento del carbón en la mezcla. y verificar si se forman las redes de carbón. Para posteriormente verificar si el material resultante es conductor, al considerar el material como una matriz donde se ha dispuesto una cierta concentración de carbón conductor. Se tratará de encontrar alguna relación entre el umbral de percolación y el tamaño de la matriz, para asi encontrar el umbral de percolación cuando el tamaño de la matriz va hacia el infinito, lo que se aproximaría al tamaño de un sistema real respecto a las partículas que lo conforman.

2. Método

Tratar de simular todas las interacciones de las partículas que envuelven el mezclado del cemento tales como la acción del agua, el grado de hidrofobia del carbón, la distribución del agua entre otros es un proceso demasiado complicado. Pero podemos hacer una aproximación de la formación de la fase de carbón fijándonos únicamente en su comportamiento y omitiendo algún otro tipo de interacción química con los otros materiales. Al ser el carbón hidrofóbica este tenderá a agruparse, entonces podemos considerar al carbón como partículas que se mueven aleatoriamente en la mezcla y cuando hacen contacto entre ellos, estos se agrupan y se mueven en conjunto, en lo que se denomina un clúster o una fase de material conductor en la mezcla consideraremos que todos los movimientos son aleatorios, y no tomaremos en cuenta la ruptura y re acoplamiento de clústeres de carbón al remover la mezcla, Para encontrar el umbral de percolación se tiene que encontrar la cantidad mínima de carbón respecto a todo el material, que debe haber para que el cemento sea conductor, y comprobar que el material es conductor debe existir un camino hecho de partículas conductoras que puedan llevar a los electrones de un extremo a otro del material. Se realizará la simulación del agrupamiento de partículas de carbón hasta que se forme una red interna o que la mayoría de las partículas están unidas. Esperamos que, debido a los movimientos aleatorios, sería conveniente empezar con bajas concentraciones de carbón en el material, ya que como son tan poco pasará demasiado tiempo en que se forma una red interna, estando sujeto a la aleatoriedad de que en algún momento finalmente sus posiciones coinciden para agruparse, por ello se partirá de una alta concentración en donde siempre se tenga una red interna que permita en un camino libre entre los extremos. Se irá bajando la concentración arbitrariamente hasta encontrar el punto en el que ya no sea conductor. Debido a que es un proceso aleatorio, se harán como mínimo 15 simulaciones cuando se considere que la concentración está cerca del umbral de percolación. Si más de 10 de las 15 simulaciones resultan conductoras, se bajará la concentración hasta que el valor sea de 10, 11 o 9. Cada vez que se varíe la concentración se hará de 1 % en 1 %, aunque reduciéndose si así se necesita. De manera que la mayoría de las veces esa concentración resultará en un conductor, a la vez que esté lo suficientemente cerca del verdadero umbral de percolación. Se realizará este proceso para diferentes volúmenes o tamaños de la matriz y verificaremos si existe alguna dependencia entre el umbral de percolación y el volumen, para así de esta manera estimar el valor del umbral de percolación cuando el tamaño de la matriz tiende al infinito, este valor refleja el umbral de percolación real del sistema debido a la cantidad de partículas que poseen los sistemas físicos reales.

2.1. Código utilizado

```
8   import numpy as np
9   import random
10  from scipy.ndimage import label
11  n=18
12  depth, rows, cols =n, n, n
13
14  np.random.seed(0)
15  matrix = np.random.choice([0, 1], size=(depth, rows, cols), p=[0.81, 0.19])
```

Figura 2: Crear la matriz con puntos aleatorios

Primero se crea una matriz donde se distribuyen aleatoriamente las partículas de material conductor. Dado que se va a cambiar frecuentemente el porcentaje de material conductor, colocamos una seed de los números aleatorios, para que de esta manera tengamos resultados reproducibles. Dado que usaremos una matriz igual en todas sus direcciones (cúbica) establecemos un único valor n que servirá para definir su tamaño. Se elegirán aleatoriamente valores entre 0 y 1 que corresponden al material

aislante y el conductor. Esto tienen una probabilidad para cada uno definida en p= [0.81,0.19], por ejemplo, aquí hay un 81 % de probabilidad de elegir 0 y un 19 % de elegir 1. Lo que representaría una concentración del 19 % de material conductor.

```
def move_cluster_3d(cluster, depth, rows, cols):
    dx, dy, dz = random.choice([-1, 0, 1]), random.choice([-1, 0, 1]), random.choice([-1, 0, 1])
    new_cluster = []
    for (z, x, y) in cluster:
        new_z, new_x, new_y = z + dz, x + dx, y + dy

        if 0 <= new_z < depth and 0 <= new_x < rows and 0 <= new_y < cols:
            new_cluster.append((new_z, new_x, new_y))
        else:
            new_cluster.append((z, x, y))
    return new_cluster
```

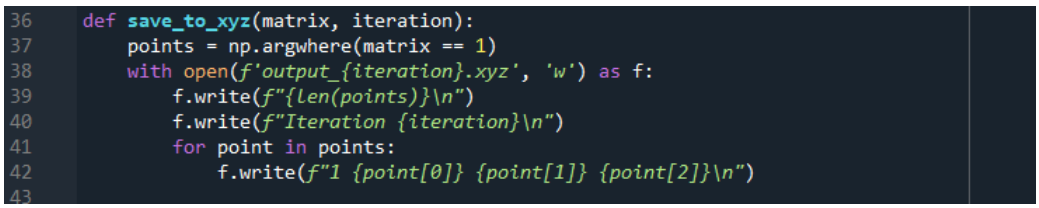
Figura 3: Mover los clústeres

La siguiente parte del código sirve para mover los clústeres dentro de la matriz, cuando los materiales conductores se juntan estos formarán un grupo que es denominado clúster. En nuestro código cada clúster es una lista de puntos que indican la posición de cada partícula conductora en la matriz. El clúster se moverá aleatoriamente en el material, recordemos que este proceso físico se da cuando mezclamos el agua con la mezcla de carbón y cemento, el cual está siendo revuelto constantemente. Para moverlo aleatoriamente se elige aleatoriamente una dirección en la que moverse, para cada dirección se elige avanzar, no moverse o retroceder. Ya que todas las partículas se mueven en conjunto, a cada una de ellas se le mueve en la misma dirección, sumándole a su posición original el valor de dz, dx y dy que representa la dirección de movimiento. Después de eso se verifica que la nueva posición de la partícula esté dentro de los límites de la matriz, si no lo está hacemos que el clúster se quede en su posición, mientras que las demás continúan su camino. Las nuevas posiciones se juntan una lista que será el nuevo clúster después de haberse movido.

```
def update_matrix_3d(matrix, clusters):
    new_matrix = np.zeros(matrix.shape)
    for cluster in clusters:
        for point in cluster:
            new_matrix[point] = 1
    return new_matrix
```

Figura 4: Actualizar la matrix con las nuevas posiciones a la matriz

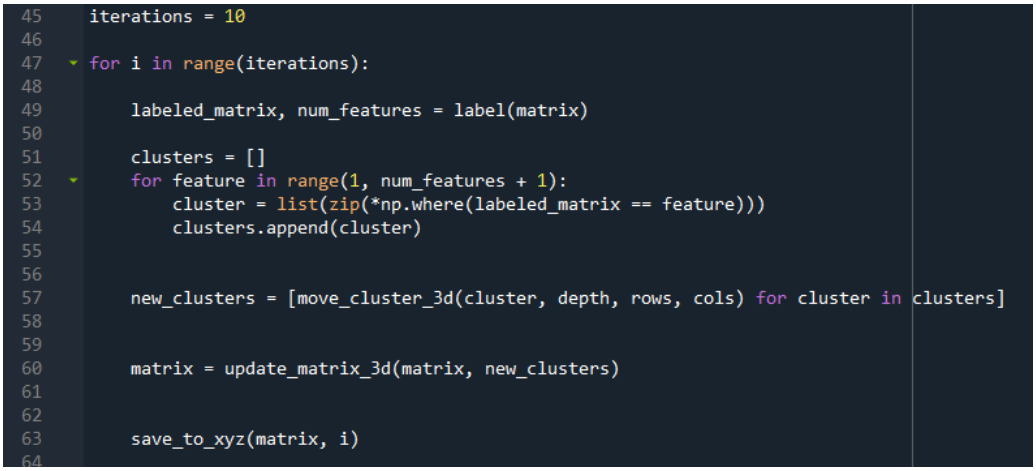
Una vez que se han movido todos los clústeres y tenemos las posiciones para cada una de las partículas hay que actualizar la matriz con las nuevas posiciones de los clústeres. Para ello se crea una matriz de ceros del mismo tamaño de la matriz original. La lista “clústeres” contiene como elementos a todos los clústeres de la matriz, y a su vez cada clúster es una lista de puntos en forma de coordenadas, entonces para cada clúster dentro de “clústeres”, para cada coordenada que almacene se va a reemplazar esa posición en la nueva matriz por un “1”, que representa al material conductor. Teniendo así la nueva matriz luego de que todos los clústeres se hayan movido. Las partículas que no se han unido a ningún grupo de partículas se consideran como un clúster con un solo elemento, y en las siguientes iteraciones, cuando quede al lado de otro clúster, este pasará a formar parte de ese clúster y se moverán en conjunto. Para cada iteración es necesario volver a identificar todos los clústeres ya que serán diferentes.



```python
def save_to_xyz(matrix, iteration):
    points = np.argwhere(matrix == 1)
    with open(f'output_{iteration}.xyz', 'w') as f:
        f.write(f"{len(points)}\n")
        f.write(f"Iteration {iteration}\n")
        for point in points:
            f.write(f"1 {point[0]} {point[1]} {point[2]}\n")
```

Figura 5: Guardar las iteraciones para visaualizarlas en OVITO

Una vez que se tiene la nueva matriz, que resulta de la interacción entre las partículas, es preferible guardarlo en un archivo para posteriormente visualizarlo en OVITO, y así, de iteración en iteración podremos ver la evolución del sistema de partículas. Esta parte del código guarda las posiciones en las cuales los elementos de la matriz es 1, es decir, la posición de los conductores, luego crea un archivo ".xyz" y primero escribe la cantidad de puntos, después el número de la iteración y finalmente la posición de cada partícula antecedido de un "1" que representa su valor.



```python
iterations = 10

for i in range(iterations):

    labeled_matrix, num_features = label(matrix)

    clusters = []
    for feature in range(1, num_features + 1):
        cluster = list(zip(*np.where(labeled_matrix == feature)))
        clusters.append(cluster)


    new_clusters = [move_cluster_3d(cluster, depth, rows, cols) for cluster in clusters]


    matrix = update_matrix_3d(matrix, new_clusters)


    save_to_xyz(matrix, i)
```

Figura 6: Identificar los clústeres y hacer las iteraciones

Una vez que se tienen las funciones necesarias se procede a hacer la simulación, se empieza definiendo cuantas iteraciones se van a realizar del movimiento y agrupamiento de las partículas. Vemos que para cada iteración se empieza usando el módulo "label". El módulo label se encarga de buscar estructuras en una matriz, considera que los valores 0 son "el fondo" y los valores que no son 0 son las partículas a identificar, se puede especificar qué tipo de estructura se desea buscar, pero al dejarlo sin esa especificación label buscará las partículas simplemente conectadas por otras adyacentes, para cada estructura o "clúster" que encuentre va a asignar un valor a sus elementos, las partículas individuales también son etiquetados. El módulo va a devolver una matriz con los clústeres etiquetados y la cantidad de clústeres encontrados, que en este caso se nombran como "labeled_matriz" y "num_features". Un ejemplo de su uso es el siguiente:

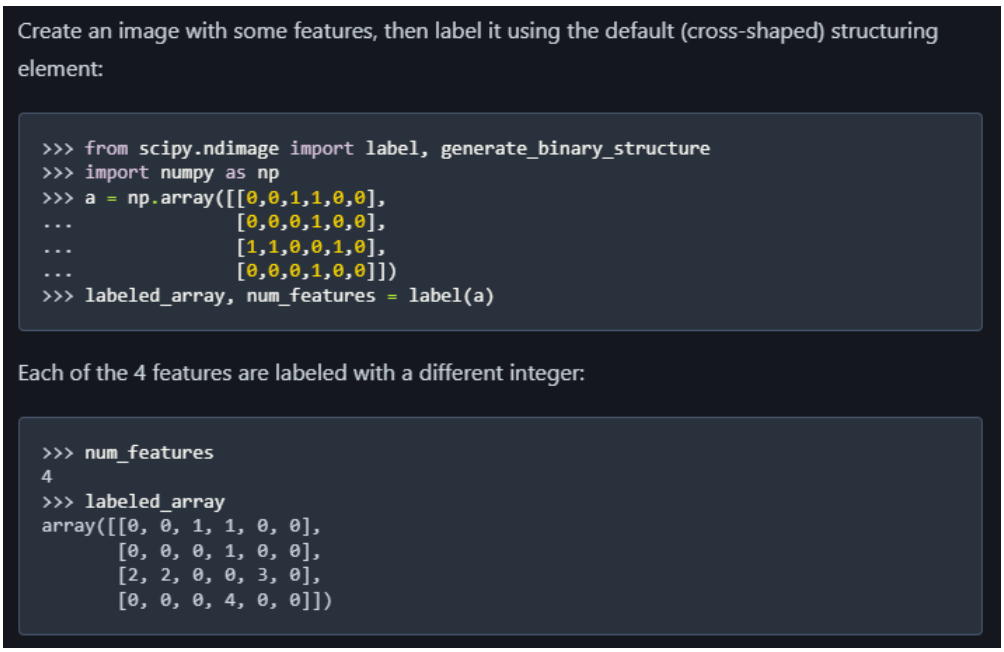


Figura 7: Ejemplo de uso de label()

Ya teniendo la matriz con los clústeres se necesita obtener la lista de las posiciones de todas las partículas que forman parte de los clústeres. Cada clúster será una lista de las partículas que forman parte de ella [SciPy, 2024] Para hacer esto se parte del primer clúster con la etiqueta de número 1. El código "labeled_matrix == feature" va a crear una matriz booleana, es decir, cuyos valores son verdaderos y falsos, y va a asignar un valor verdadero cuando se cumpla que el elemento de la matriz es igual al de la etiqueta "feature", es decir 1 en el primer clúster, por ejemplo. El comando np.where va a devolver las posiciones de en donde los valores de la matriz son verdaderos, sin embargo lo va a hacer devolviendo solamente una lista con tres arrays, una para las x, otra para las y otra para las z. [Numpy.org, 2024] Para poder obtener una lista de elemento en la que cada elemento represente una posición se necesita primero añadir "*" antes del comando, al hacer esto el comando va a devolver dos arrays por separado. Los dos arrays sirven como argumento para el comando "zip" el cuál toma el primer valor de cada array y las junta en una lista, hace lo mismo con el segundo, con el tercero y así sucesivamente, para poder obtener la lista del comando "zip" es necesario colocar "list" antes, ya que por sí solo no lo va a hacer, sino que devuelve una dirección de la memoria RAM donde se almacenó la información[GeeksforGeeks, 2024] Una vez obtenida la lista de puntos se le asigna al valor clúster, y a su vez este se añade a una lista mayor llamada "clústeres" que almacena todos los clústeres. Se crean los nuevos clústeres con la función definida anteriormente y se actualiza la matriz. se guarda la matriz en un archivo para visualizarlo en OVITO y repiten las iteraciones necesarias.

```python
def is_connected_central(matrix):
    visited = np.zeros_like(matrix, dtype=bool)
    def dfs(i, j, k):
        if not (4 <= i < n-4 and 4 <= j < n-4 and 4 <= k < n-4):
            return False
        if visited[i, j, k] or matrix[i, j, k] != 1:
            return False
        visited[i, j, k] = True
        if i == n-5:
            return True

        return (dfs(i-1, j, k) or dfs(i+1, j, k) or dfs(i, j-1, k) or
                dfs(i, j+1, k) or dfs(i, j, k-1) or dfs(i, j, k+1))


    for j in range(4, n-4):
        for k in range(4, n-4):
            if matrix[4, j, k] == 1 and dfs(4, j, k):
                return True
    return False

connected = is_connected_central(matrix)
print("¿El material es conductor?", connected)
```

Figura 8: Código de búsqueda en profuncidad para buscar caminos

Finalmente, y más importante se procede a verificar si el material es conductor. Para saber si es conductor debe haber un camino de partículas conductoras en contacto directo de manera que se pueda ir de un extremo a otro de la matriz, estos caminos servirían como vías en donde los electrones se trasladan. El código empieza creando una matriz de tipo booleana del mismo tamaño que la matriz del que se va verificar. Esta matriz al ser del tipo booleana va a tener como únicos valores posibles verdadero o falso. Las posiciones que son verdaderas en esta matriz van a representar los puntos en los que ya se "pasado" al buscar un camino, esto para evitar bucles y redundancias. Luego se crear una función que va a buscar los caminos a partir de un punto inicial. Primero se comprueba si el punto está dentro de la matriz, si no lo está la función de buscar caminos va a regresar un valor de "falso", luego verifica si ese punto ya ha sido visitado y si el punto no contiene material conductor, si alguno de ellos se cumple volverá a retornar un valor de falso. Si el punto está dentro de la matriz, no ha sido visitado y es conductora, entonces a esa posición en la matriz booleana se le asigna el valor de verdadero, señalándolo como un lugar visitado. Luego comprueba si la coordenada i del punto representa un punto en la otra cara de la matriz, si esto se cumple retornará un valor de verdadero, de lo contrario esta función se llamará a sí misma para los puntos adyacentes al que se trabajaba. Como vemos es posible que en la definición de la función esta se llame a sí misma a pesar de haberse terminado de definir. Como vemos esta función verifica si el punto es conductor y si el punto está en la cara opuesta, si no lo está, procederá a repetir el proceso con los puntos alrededor excluyendo los ya visitados. Cuando verifique un punto adyacente procederá a verificar el adyacentes este y así sucesivamente hasta llegar al otro extremo. Basta con que alguno de las tantas funciones resulte ser verdadera, al ser parte de otra función de un punto anterior, su valor será retornada hacia la función superior, y a su vez este retornará su función verdadera al superior, así hasta que la función principal sea verdadera y finalmente se verifique que sí existe algún camino que llegue al otro lado de la matriz. Esta función se aplica a cada punto existente en la primera cara de la matriz, y si alguno es verdadero la función "is_conected_central" será verdadera, de lo contrario, falsa. Para visualizar los resultados se usó el software OVITO, que permite visualizar simultánea y detalladamente las diferentes iteraciones de las simulaciones, y además no dar problemas de rendimiento en el caso de esta simulación frente a otros programas no dedicas a ello como Spyder. Para visualizar las matrices se tuvieron que seleccionar los archivos ".xyz" para cada iteración y se pudieron ver como parte de una misma simulación en el software.

3. Resultados

Se realizaron simulaciones para tamaño de matrices de tamaño de lado n=10, 15, 20, 25, 30, 35, 40, 45 y 50 partículas. No se realizaron para tamaños más grandes debido a que el tiempo para obtener el resultado se alarga enormemente, yendo desde varios minutos para n=60, y no obteniéndose resultado alguno para n=100 después de una hora y medio de espera, tiempo que extendería aún más por la cantidad de simulaciones que se deben hacer para calcular el umbral. Para un n=28 se obtuvieron las siguientes iteraciones para una concentración del 40 %.

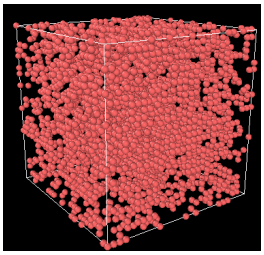


Figura 9: Iteración 1

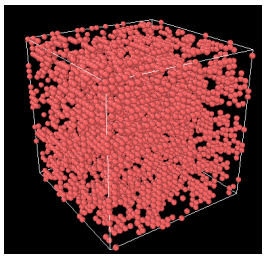


Figura 10: Iteración 2

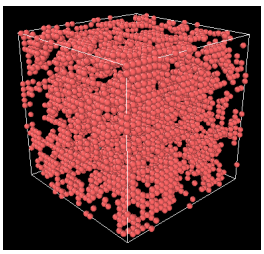


Figura 11: Iteración 3

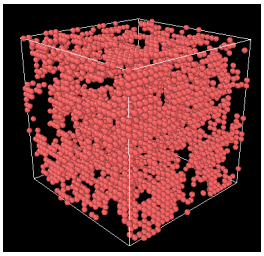


Figura 12: Iteración 4

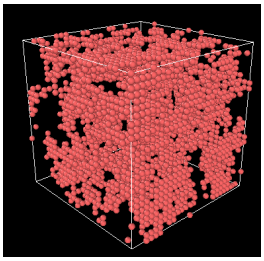


Figura 13: Iteración 5

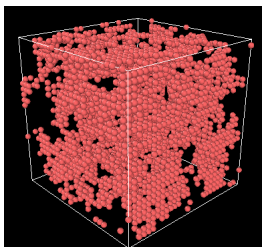


Figura 14: Iteración 6

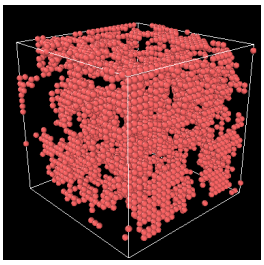


Figura 15: Iteración 7

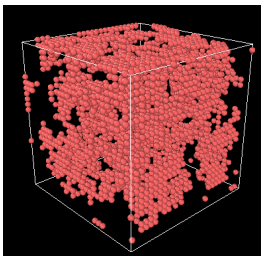


Figura 16: Iteración 8

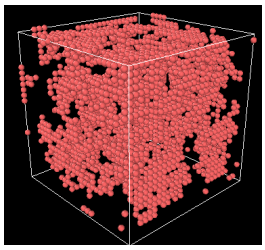


Figura 17: Iteración 9

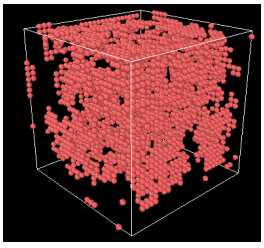


Figura 18: Iteración 10

Podemos ver que la red de partículas conductoras se forma rápidamente, con tan solo 4 iteraciones. Las iteraciones sucesivas son de la estructura moviéndose como una sola en el interior de la matriz, pero debido a que las partículas de los extremos se quedan en su sitio y las otras siguen avanzado (a pesar de estar en un mismo clúster, como ha sido programado) esto resulta en que la red de partículas esté alejándose de las paredes de la matriz y apegándose a otras. Una vista superior muestra mejor este hecho.

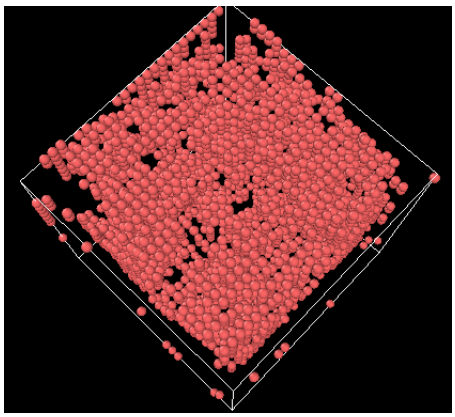


Figura 19: Vista superior de la última iteración

Esto hacía que en repetidas ocasiones el código que verifica si es conductor busca los caminos llegué a un resultado falso a pesar de tener una considerable concentración. Por ello se cambió el tamaño de la matriz por una que tenga un tamaño mayor de n+8, y la verificación se realizó en una matriz centrada que incluiría a las 4 primeras capas externas de la matriz mayor. De esta manera la simulación es más precisa ya que en la realidad no habrá paredes o bordes de tal magnitud que impidan el movimiento de las partículas. Así para la matriz de tamaño n=10 se simuló una de n=18, para n=15 un n=23 y así sucesivamente. Para cada tamaño de matriz se verificó constantemente que el número de iteraciones fuera necesaria para que se empiece a formar un sólo clúster que abarque a la mayoría de partículas, en todos los casos este es alcanzaba en pocas iteraciones, aumentando hasta 13 iteraciones a medida que se aumenta el tamaño de la matriz. Los resultados del umbral de percolación de listaron en la siguiente tabla.

Tamaño de matriz nxnxn	Umbral de percolación
10	30%
15	25%
20	19%
25	14.5%
30	13%
35	12.5%
40	11%
45	10.25%
50	10%

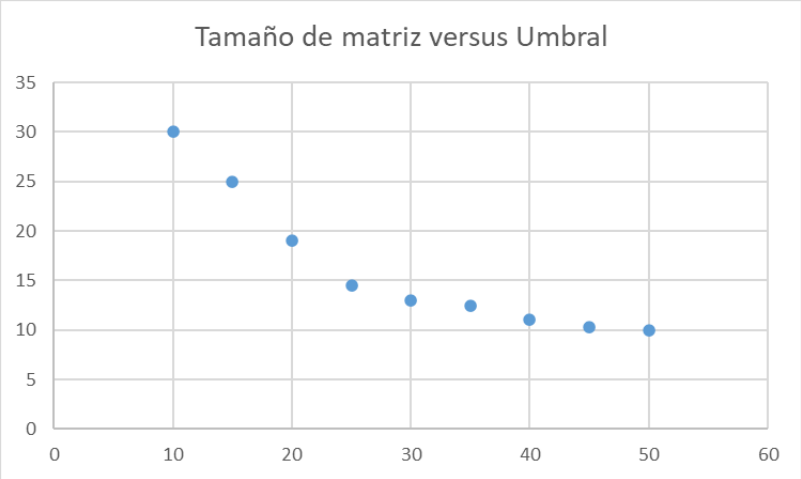


Figura 20: Valores de umbral de percolación, el eje y es el umbral y el eje x es el tamaño n de la matriz

Dado que se trata de un proceso aleatorio existen inexactitudes en los valores, pero aun así es posible notar claramente un comportamiento exponencial decreciente. Podríamos intentar explicarlo como que si la matriz es de un tamaño de n=1, solo se necesitará una concentración del 100 % para volverlo conductor, si es de 2x2x2, se tendrán 8 casillas y se necesitarán como mínimo 2, es decir un 25 % de concentración. A la vez que la matriz se hace inmensa, aunque la concentración sea del 1 % este 1 % podría agruparse en una línea que fácilmente llegaría al otro extremo, llegando a un valor de 0 cuando n está en el infinito. En el artículo "Carbon–cement supercapacitors as a scalable bulk energy storage solution" se menciona que "Our material design departs from previous investigations of the electrical conductivity of carbon-cement composites and their resistive heating potential (Joule effect) (15–17), which was attributed to the development of a carbon network at a percolation threshold of a mere 3 to 4 Vol %(17)". El umbral de percolación encontrado está entre el 3 % y el 4 %, datos que se aproximan a la gráfica encontrada considerando que el n sería muy grande pero no infinita.

4. Conclusiones

Se determinó el comportamiento del umbral de percolación según el volumen del medio en el que está. Se mostró un código que simula el movimiento aleatorio de las partículas de un conductor en un medio, las partículas se agrupan cuando se encuentran con otras formando clústeres, los clústeres también se mueven aleatoriamente encontrándose con otros clústeres o partículas. Posteriormente comprueba la conductividad del material resultante de la combinación del medio con el conductor, para ello busca un camino hecha de partículas conductoras adyacentes que inicie en un lado de la matriz y termine en el lado opuesto. Se realizaron simulaciones para matrices de tamaño nxnxn desde n=10 aumentando de 5 en 5 hasta n=50. Debido al movimiento de los clústeres con las paredes de la matriz, estos se apegaban a uno de los lados dejando sin contacto a la otra, alejándose del fenómeno físico real y dando resultados incorrectos en la búsqueda de caminos. Por ello se aumentó el tamaño de las matrices para realizar la búsqueda en un matriz centrada que ignore las capas externas. Los resultados del umbral de percolación muestran que tienen un comportamiento exponencial decreciente a medida que el tamaño de la matriz aumenta. El número de iteraciones para alcanzar una red interna también aumenta con el tamaño de la matriz. Para n=50 el umbral es de solo 10 %, mientras que para n=10 es de 30 %. Estos resultados concuerdan con los datos experimentales encontrados en otro artículo.

Referencias

[Chanut et al., 2023] Chanut, N., Stefaniuk, D., Weaver, J. C., Zhu, Y., Shao-Horn, Y., Masic, A., and Ulm, F.-J. (2023). Carbon–cement supercapacitors as a scalable bulk energy storage solution. *Proceedings of the National Academy of Sciences*, 120(32):e2304318120.

[Conductive, 2021] Conductive, S.-H. E. (2021). Self-heating electrically conductive concrete demonstration project.

[GeeksforGeeks, 2024] GeeksforGeeks (2024). zip in python. https://www.geeksforgeeks.org/zip-in-python/. Accedido el: 9 de julio, 2024.

[Numpy.org, 2024] Numpy.org (2024). numpy.where. https://numpy.org/doc/stable/reference/generated/numpy.where.html. Accedido el: 9 de julio, 2024.

[Obregón G. Jesús, 2019] Obregón G. Jesús, García P. Georgina, M. C. J. E. (2019). Algoritmo para determinar umbral de percolación en compuestos poliméricos con nanopartículas metálicas embebidas: Un enfoque aproximado. *CNMN-IPN y ESIME UPT-IPN*.

[Savytskyi et al., 2024] Savytskyi, M., Sukhyy, K., Savytskyi, O., Babenko, M., and Shevchenko, T. (2024). Carbon materials for electrically conductive concrete. In *E3S Web of Conferences*, volume 534, page 01019. EDP Sciences.

[SciPy, 2024] SciPy (2024). label. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.label.html. Accedido el: 9 de julio, 2024.